MINI PROJECT REPORT

ON
CURRENCY CONVERTER

SUBMITTED BY
PRAGIIN:B
NC.SC.U4CSE24030
FOR
23CSE111-OBJECT ORIENTED PROGRAMMING
BTECH CSE-A

INTODUCTION:

In today's globalized economy, currency conversion plays a crucial role in international trade, travel, and finance. A currency converter is a tool that allows users to convert the value of one currency into another based on real-time exchange rates. This project/application provides a simple and efficient way to perform currency conversions, ensuring accuracy and ease of use. Whether for business purposes, personal travel planning, or online shopping, a currency converter helps users make informed financial decisions by providing up-to-date exchange rate information.

Problem Statement:

In an increasingly interconnected world, individuals and businesses frequently need to convert one currency into another for purposes such as international travel, online shopping, or financial transactions. However, manual conversion using fluctuating exchange rates is time-consuming, error-prone, and inconvenient.

The objective of this project is to develop a **desktop-based Currency Converter application** using Java Swing that enables users to quickly and accurately convert between different currencies. The application should provide a user-friendly graphical interface where users can:

- Select source and target currencies
- Enter the amount to be converted
- View the converted result instantly

The system should use predefined static exchange rates (stored in a HashMap) and be easily extendable to include more currencies or integrate real-time exchange rate APIs in the future.

Objectives

1. **To develop a user-friendly GUI application** using Java Swing for converting currency values between multiple international currencies.
2. **To provide accurate currency conversion** by using predefined static exchange rates stored in a `HashMap`.
3. **To allow users to input an amount**, select source and target currencies, and view the converted result immediately.
4. **To handle user input errors** (such as non-numeric values) gracefully with appropriate error messages.
5. **To create a modular and scalable design**, allowing easy extension of the application to support more currencies or integration with live exchange rate APIs in the future.
6. **To enhance understanding of event-driven programming** and GUI design in Java for educational or practical software development purposes.

UML DIAGRAM OF THIS PROJECT

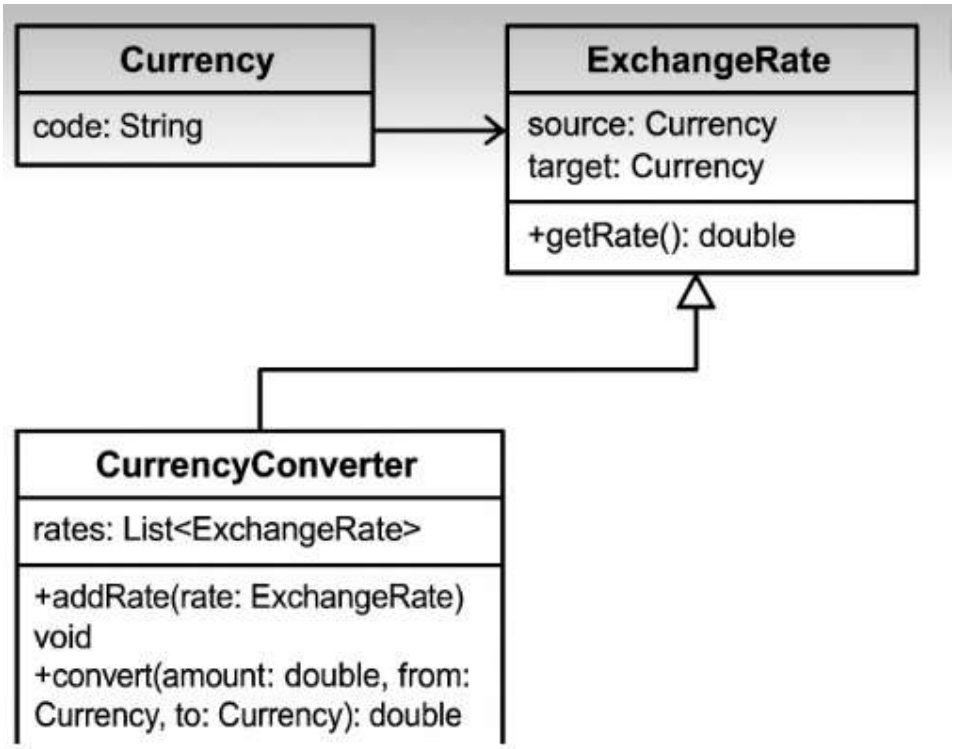
1.CLASS DIAGRAM

2.OBJECT DIAGRAM

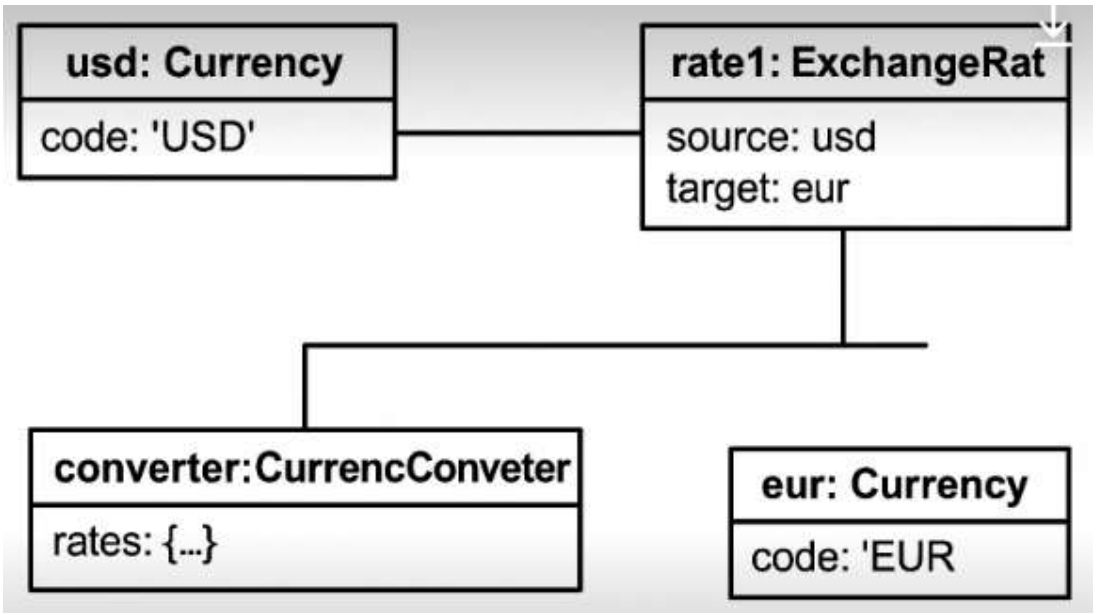
3.USE CASE DIAGRAM

4.ACTIVITY DIAGRAM

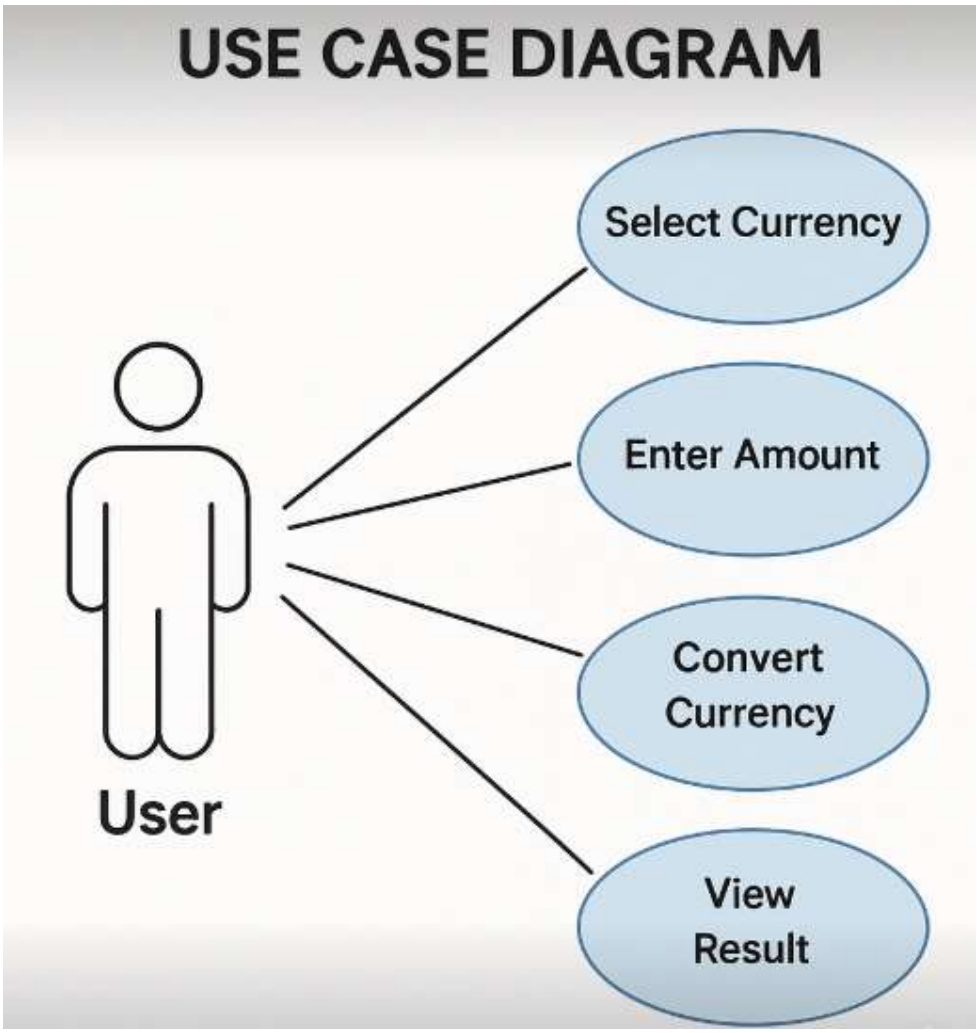
CLASS DIAGRAM:



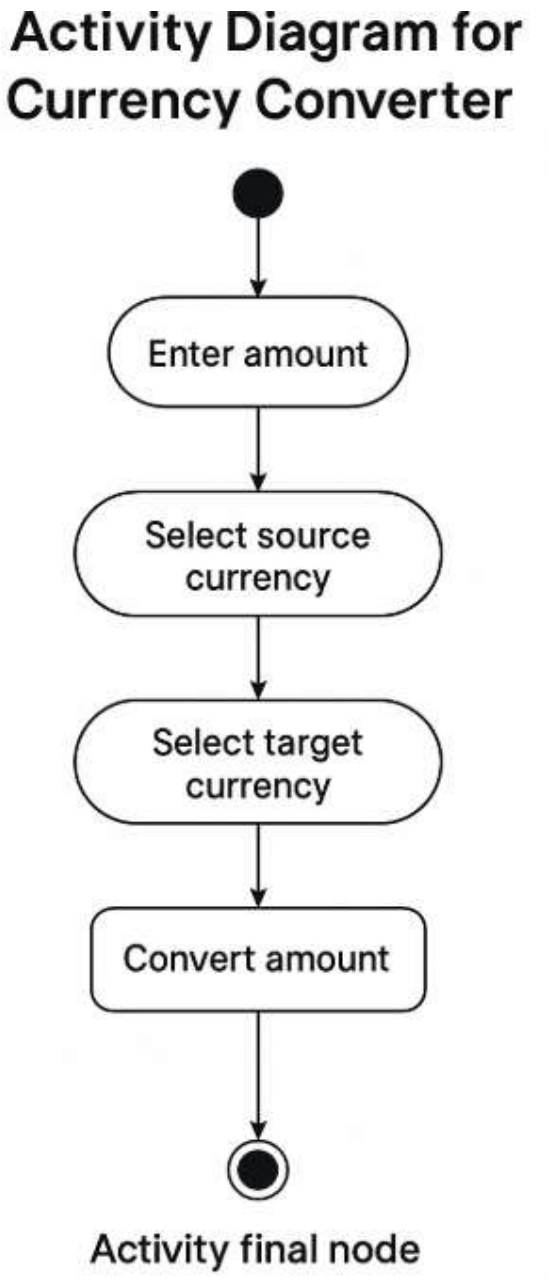
OBJECT DIAGRAM:



USE CASE DIAGRAM



ACTIVITY DIAGRAM:



MODULES OF THE PROJECT:

1. User Interface Module

- Handles the GUI using **Swing components** (`JFrame`, `JLabel`, `JComboBox`, `JTextField`, etc.).
- Provides dropdowns for currency selection and a field for entering amounts.
- Displays conversion results and error messages.

2. Currency Data Module

- Manages predefined **exchange rates** using a `HashMap`.
- Allows easy modification or extension for integrating real-time exchange rate APIs.

3. Conversion Logic Module

- Implements methods to fetch exchange rates.
- Performs **currency conversion calculations**.
- Ensures proper handling of edge cases like **same currency selection**.

4. Error Handling Module

- Validates user input to prevent invalid conversions.
- Displays meaningful error messages if users enter **non-numeric values**.

5. Event Handling Module

- Listens for button clicks (`ActionListener` implementation).
- Triggers conversion calculations when the "Convert" button is clicked.

6. Future Enhancement Module (Optional)

- Integrate **live API** for real-time exchange rates.
- Implement **multi-language support** for better global accessibility.
- Add **historical exchange rate analysis** for financial insights.

CODE:

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

import java.util.HashMap;

import java.util.Map;


public class CurrencyConverterApp extends JFrame {

   private final String[] currencies = {"USD", "EUR", "INR", "GBP",
"JPY"};

   private JComboBox<String> fromBox, toBox;

   private JTextField amountField;

   private JLabel resultLabel;


   // Exchange rates stored statically (you can replace this with live
API)

   private static final Map<String, Double> rates = new HashMap<>();


   static {

      rates.put("USD_INR", 83.2);

      rates.put("INR_USD", 0.012);

      rates.put("USD_EUR", 0.93);
```

```
        rates.put("EUR_USD", 1.07);
        rates.put("USD_GBP", 0.80);
        rates.put("GBP_USD", 1.25);
        rates.put("USD_JPY", 154.5);
        rates.put("JPY_USD", 0.0065);
        rates.put("EUR_INR", 89.4);
        rates.put("INR_EUR", 0.011);
        // Add more currency pairs as needed
    }

    public CurrencyConverterApp() {
        setTitle("Currency Converter");
        setSize(450, 250);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new GridBagLayout());
        setLocationRelativeTo(null);

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(8, 8, 8, 8);
        gbc.fill = GridBagConstraints.HORIZONTAL;
```

```
        fromBox = new JComboBox<>(currencies);
        toBox = new JComboBox<>(currencies);
        amountField = new JTextField();
        resultLabel = new JLabel("Converted Amount: ");
        JButton convertBtn = new JButton("Convert");

        gbc.gridx = 0; gbc.gridy = 0; add(new JLabel("From Currency:"),
gbc);
        gbc.gridx = 1; gbc.gridy = 0; add(fromBox, gbc);
        gbc.gridx = 0; gbc.gridy = 1; add(new JLabel("To Currency:"),
gbc);
        gbc.gridx = 1; gbc.gridy = 1; add(toBox, gbc);
        gbc.gridx = 0; gbc.gridy = 2; add(new JLabel("Amount:"), gbc);
        gbc.gridx = 1; gbc.gridy = 2; add(amountField, gbc);
        gbc.gridx = 1; gbc.gridy = 3; add(convertBtn, gbc);
        gbc.gridx = 1; gbc.gridy = 4; add(resultLabel, gbc);

        convertBtn.addActionListener(e -> convert());

        setVisible(true);
    }
```

```
    private void convert() {
        try {
            double amount = Double.parseDouble(amountField.getText());
            String from = (String) fromBox.getSelectedItem();
            String to = (String) toBox.getSelectedItem();

            double rate = getRate(from, to);
            double result = amount * rate;
            resultLabel.setText("Converted Amount: " +
String.format("%.2f", result) + " " + to);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Please enter a valid
numeric amount.", "Input Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    private double getRate(String from, String to) {
        if (from.equals(to)) return 1.0;
        return rates.getOrDefault(from + "_" + to, 1.0); // Default to 1 if
no rate found
    }
```

```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(CurrencyConverterApp::new);
    }
}
```

OUTPUT SREENSHOT:

APPLICATION OF THE PROJECT:

1. Financial Transactions

- Helps users **convert currencies** when making international transactions.
- Useful for banking applications, remittance services, and online payment gateways.

2. E-Commerce & Global Trade

- Assists online shoppers in seeing product prices in **local currency**.
- Facilitates currency conversion for **cross-border purchases** on e-commerce platforms.

3. Travel & Tourism

- Allows travelers to check **real-time conversion rates** for expenses abroad.
- Can be integrated into travel apps or hotel booking platforms.

4. Stock Market & Investment Analysis

- Helps investors analyze **foreign exchange rates** when dealing with global investments.
- Useful for forex traders in monitoring **currency fluctuations**.

5. Education & Learning

- Can be used as an educational tool to teach students about **currency exchange and finance**.
- Helps learners understand financial mathematics concepts in economics courses.

6. Business and Accounting

- Assists accountants and businesses in managing **multi-currency transactions**.
- Supports companies operating in **multiple countries** with financial reporting.

LIMITATION OF THE PROJECT:

1. Static Exchange Rates

- The project currently uses **hardcoded exchange rates** stored in a `HashMap`. This means the rates won't reflect real-time fluctuations in currency values.
- Solution: Integrate an **API** (e.g., Open Exchange Rates or Forex API) for live exchange rate updates.

2. No Historical Data Support

- Users can convert currencies only for the **current rate** but cannot view historical trends.
- Solution: Add a **database** or external service to track historical exchange rates and allow comparison.

3. Limited Currency Options

- The project supports only **a handful of currencies**.
- Solution: Expand the range of currencies supported or allow **custom input** for less common currencies.

4. Single Conversion at a Time

- The application performs **one-to-one conversions**, limiting batch or multi-currency conversions.
- Solution: Introduce **bulk conversion** and multi-currency comparison features.

5. No Network-Based Conversion

- The application doesn’t interact with external services; all conversions are **local**.
- Solution: Enhance it with **web-based conversion** where users get updated rates via online services.

6. No Mobile or Web Interface

- The project is **Swing-based**, meaning it runs as a desktop application.
- Solution: Develop a **web-based version** using React, Angular, or Flask, or create a **mobile app** for better accessibility.

BIBLIOGRAPHY

- **API Documentation**
  - Open Exchange Rates API: https://openexchangerates.org
  - Forex API: https://www.forexapi.com
- **Web Development & Frameworks**
  - React Documentation: https://react.dev
  - Flask Web Framework: https://flask.palletsprojects.com
- **Java & Swing Resources**
  - Java SE Documentation: https://docs.oracle.com/javase
  - Java Swing Tutorial: https://docs.oracle.com/javase/tutorial/uiswing
- **Currency & Financial References**
  - International Monetary Fund (IMF): https://www.imf.org
  - World Bank Currency Data: https://data.worldbank.org
- **UML & System Design**
  - UML Diagrams & Object-Oriented Design: https://www.uml-diagrams.org